

Dati del progetto

- **Studente:** Christian Cipriano e Diego Anostini
- **Classe:** 3[^]BTLC
- **Titolo del gioco:** Giorgi Tower Defense

Idea del gioco

- **Tipologia:** Si tratta di un videogioco strategico di tipo Tower Defense bidimensionale, sviluppato in Python tramite la libreria Pygame.
- **Obiettivo del giocatore:** Il giocatore deve difendere l'integrità della scuola posizionando strategicamente delle torrette (rappresentate dai professori Fiorito, Lopez e Quirino) lungo la mappa per eliminare le ondate di truppe nemiche (gli studenti e i professori Boss) prima che raggiungano la fine del percorso scenico.
- **Elementi di originalità:** Il gioco è interamente ambientato all'interno dell'Istituto Tecnico "Giorgi". Lo sfondo riproduce un pavimento scolastico con banchi, sedie e una lavagna con formule ironiche. I "nemici" da sconfiggere sono caricature dei professori storici dell'istituto (Dagnoko, Trioschi, Cascone, Mattana, Musci) che avanzano lanciando sotto-truppe a tema scolastico (libri, cavi, formule matematiche, codici sorgente).
- **Pubblico destinato:** Compagni di classe, studenti ed ex-studenti dell'istituto appassionati di giochi di strategia e dinamiche arcade.

Analisi e progettazione

Regole del gioco

- **Condizioni di sconfitta:** Il giocatore parte con 100 punti di "Integrità Scuola". Ogni studente semplice che supera il percorso sottrae 4 punti vita; il passaggio di un insegnante "Boss" ne sottrae ben 25. Se l'integrità scende a 0, si visualizza la schermata di Game Over ("BOCCIATO!").
- **Condizioni di vittoria:** Sopravvivere a tutte le 25 ondate di gioco e sconfiggere l'ultimo esame rappresentato dal boss finale (Prof. Musci). La vittoria mostra la schermata "PROMOSSO!".
- **Economia di gioco:** Eliminando i nemici si ottengono dollari (\$) scolastici. La valuta serve per comprare nuove torri o per potenziarle fino al livello 3. Cliccando con la rotellina del mouse è possibile vendere una torre recuperando il 50% dell'investimento totale.
- **Meccaniche speciali:** Premendo la BARRA SPAZIATRICE il gioco accelera a 5X per velocizzare i tempi morti. Tenendo premuto il tasto E vengono mostrati visivamente a schermo i raggi d'azione di tutte le torrette piazzate.

Struttura del programma

Il codice è strutturato all'interno di un unico script principale basato su un ciclo di gioco (game loop) continuo che gestisce gli eventi, l'aggiornamento logico delle posizioni e il rendering grafico. Le librerie utilizzate sono tre: pygame (gestione di grafica, finestre, font e input), math (calcolo delle distanze euclidee e delle funzioni trigonometriche per le orbite dei boss e dei proiettili) e random (generazione casuale dei voti sui banchi e del bottino in denaro).

Flusso logico del programma (Pseudocodice):

- **All'avvio:** Inizializzazione di Pygame, impostazione della finestra (800x600) e posizionamento casuale degli elementi scenici (banchi ed ostacoli). Mostra lo stato MENU.
- **Nel ciclo principale (while running):**
 - Se lo stato è MENU/PAUSE: Resta in ascolto dei click sui pulsanti di avvio, ripresa o chiusura.
 - Se lo stato è PLAYING: * Legge gli input di mouse e tastiera (gestione acquisti, upgrade, vendita, accelerazione e raggi d'azione).
 - Controlla la logica del timer delle ondate: genera i nemici semplici o i Boss specifici se l'ondata è un multiplo di 5.
 - Aggiorna la posizione dei nemici lungo i nodi del percorso (PATH) e muove i proiettili verso i bersagli.
 - Verifica le collisioni: se un proiettile colpisce il bersaglio, riduce la vita del nemico e calcola l'eventuale rimbalzo (ricochet).
 - Disegna lo sfondo (griglia stile quaderno), il percorso grigio, i banchi, i nemici con le barre della vita, le torri e l'interfaccia utente (UI) dello shop.
- **Alla chiusura:** Gestione della sconfitta o vittoria con interruzione del ciclo e disattivazione dei moduli.

Strutture dati importanti

Essendo un paradigma orientato agli oggetti, sono state implementate delle Classi per modellare le entità dinamiche:

- **Classe Enemy:** Rappresenta i nemici. Contiene attributi fondamentali come posizione (**x, y**), velocità, raggio di collisione, vita attuale/massima e variabili specifiche per identificare se si tratta di un'unità standard, di un "Boss" o di una sotto-truppa (es. tipo "cavo" o "formula").
- **Classe Tower:** Rappresenta le postazioni difensive. Memorizza le coordinate, il livello di upgrade (1, 2 o 3), il tempo di ricarica (cooldown), il danno inflitto e il raggio di tiro. Possiede metodi per calcolare il costo del potenziamento e per applicare i potenziamenti passivi (l'area di efficacia della torre Lopez aumenta il danno delle torri vicine).
- **Classe Bullet:** Gestisce i singoli proiettili attivi a schermo. Tiene traccia del punto di partenza, del puntatore all'oggetto nemico da inseguire, della velocità e del flag di rimbalzo per colpire un secondo bersaglio vicino.
- **Dizionario banchi_posic:** Associa le coordinate (x, y) simulate sulla mappa a una stringa di testo che rappresenta un voto insufficiente (es. "3", "4"). Serve a evitare il posizionamento delle torri sopra i banchi.

Sviluppo e uso dell'AI

Organizzazione del lavoro

Il lavoro è stato condotto seguendo una scaletta rigorosa:

1. **Fase ideativa:** Scrittura dei requisiti e scelta dell'ambientazione scolastica.

2. **Sviluppo della base:** Creazione del sistema di movimento dei nemici lungo i punti prefissati del percorso.
3. **Integrazione delle torri:** Scrittura del sistema di puntamento geometrico ed implementazione dello shop.
4. **Rifinitura estetica e bilanciamento:** Aggiunta dei dettagli visivi e calibrazione dei costi d'acquisto.

Le difficoltà maggiori si sono riscontrate nella gestione matematica delle traiettorie dei proiettili (evitando che rimanessero sospesi se il nemico moriva prima dell'impatto) e nel prevenire la sovrapposizione delle torrette sopra il percorso stradale o sopra i banchi della scuola.

Utilizzo consapevole dell'intelligenza artificiale

L'assistente virtuale è stato utilizzato come supporto tecnico interattivo, principalmente per tre scopi:

- **Generazione della logica geometrica:** Richiesta di algoritmi matematici per determinare la distanza minima tra il mouse ed un segmento per impedire il posizionamento delle torri sulla strada (**dist_point_to_segment**).
- **Risoluzione di problemi di visibilità (Bug di contrasto):** Durante i test, le scritte testuali di stato delle truppe superiori (come l'indicatore **"RICOCHET"** delle torri di livello 3) erano renderizzate in verde scuro, risultando completamente invisibili quando i nemici transitavano sopra lo sfondo grigio del percorso o sul bianco del pavimento. È stato chiesto all'AI di strutturare una funzione per applicare un contorno ad alto contrasto. L'AI ha proposto la funzione personalizzata **draw_text_with_outline**
- **Esempi di prompt utilizzati:**
"Crea una funzione in Pygame che disegni un testo con un contorno (outline) nero di 1 pixel attorno alle lettere, in modo che sia leggibile sia su sfondo bianco che su sfondo grigio scuro."
"Come posso calcolare se un cerchio si trova sopra una linea spezzata composta da una lista di punti?"

Il codice fornito dall'AI non è stato copiato passivamente: è stato necessario adattare i riferimenti geometrici alle coordinate globali del gioco e modificare i colori dell'outline per fare in modo che le scritte dei Boss mantenessero un aspetto pulito senza rallentare il frame rate complessivo dovuto ai continui cicli di rendering del testo.

Test e risultati

Casi di test effettuati

Per verificare la stabilità dell'applicazione sono stati eseguiti dei test sistematici riportati nella tabella seguente:

Test	Azione eseguita	Comportamento atteso	Esito riscontrato

1	Clic sinistro sul percorso stradale	Il sistema deve bloccare il posizionamento della torre	Superato (La torre non viene creata)
2	Raggiungimento di 0 punti Integrità	Interruzione del loop e avviso di sconfitta	Superato (Schermata BOCCIATO attiva)
3	Transito delle truppe "RICOCHET" su sfondo misto	Il testo deve rimanere leggibile sia sul grigio che sul bianco	Superato (L'outline nero evidenzia perfettamente i caratteri)
4	Clic con tasto destro su torre Livello 3	Nessun addebito economico e nessun aumento di livello	Superato (Il tetto massimo viene rispettato)

Bug identificati e soluzioni provvisorie

Un problema riscontrato durante lo sviluppo riguardava i proiettili legati alla meccanica di rimbalzo: se il secondo nemico selezionato dal proiettile muore un istante prima del contatto a causa di un'altra torre, il proiettile continua a viaggiare all'infinito fuori dai confini dello schermo. Come soluzione provvisoria, è stato inserito un controllo di validità (**if self.target not in enemies**): se il bersaglio scompare dalla lista globale dei nemici attivi, il proiettile viene rimosso forzatamente dalla memoria.

Conclusioni e riflessione personale

L'esperienza di sviluppo di Giorgi Tower Defense ha permesso di comprendere a fondo le dinamiche di gestione dei videogiochi bidimensionali e l'importanza della pianificazione logica delle strutture dati. Lo studio autonomo delle Classi in Python (argomento non ancora affrontato formalmente in classe) si è rivelato indispensabile per mantenere il codice organizzato, flessibile ed espandibile.

L'utilizzo dell'AI è stato affrontato in modo critico: lo strumento è servito per superare i blocchi algoritmici legati alla geometria analitica e per risolvere bug grafici complessi (come la leggibilità dei testi tramite outline), ma la logica del gioco, il bilanciamento dei prezzi, l'assegnazione dei punteggi e la personalizzazione dell'ambientazione sono stati ideati e supervisionati interamente dallo sviluppatore. Se dovessi ricominciare il progetto, opterei per una suddivisione del codice in moduli separati (file distinti per le classi, per la gestione dei menu e per i dati delle ondate) per migliorare ulteriormente la manutenibilità dello script.

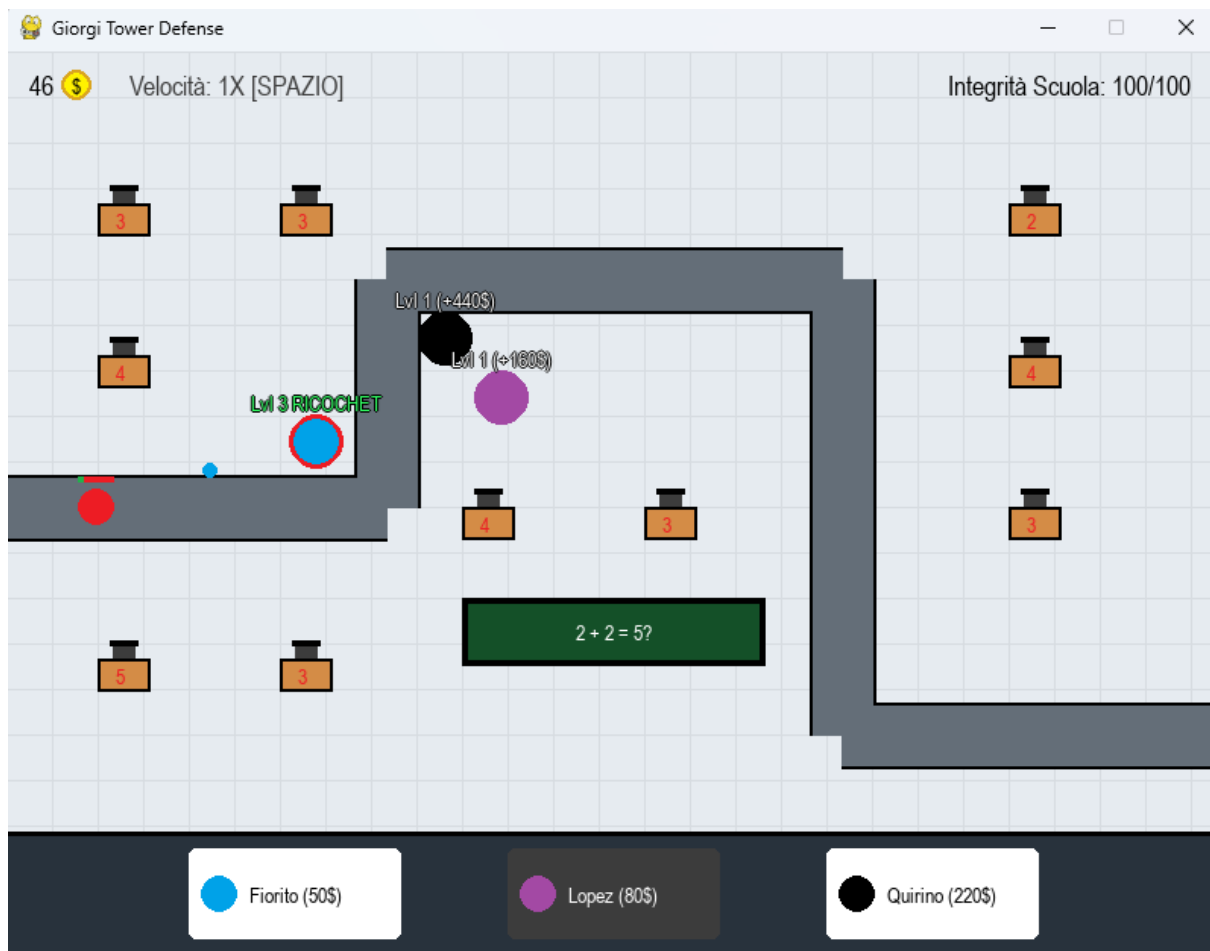
Allegati

Nota per la stampa: In questa sezione finale della relazione cartacea verranno allegati gli screenshot delle schermate di gioco (Schermata iniziale del Menu, fase di combattimento attivo con l'interfaccia dello Shop visibile in basso e schermata finale di Game Over).

MENU PRINCIPALE



FASE DI COMBATTIMENTO



BOCCIATO oppure PROMOSSO

BOCCIATO!

PROMOSSO! MATURITÀ SUPERATA!